

Jenkins challenges 2

Create a Declarative Jenkins parameterized pipeline for simplecustomerapp.

Requirements:

- Create a Pipeline job in Jenkins.
- Use a Git repository for simplecustomerapp.
- Add parameters:
 - `BRANCH_NAME` (string, default: main)
 - `ENV` (choice: dev, qa)
- Define environment variables:
 - `APP_NAME` = simplecustomerapp
- Implement stages:
 - Checkout (based on `BRANCH_NAME`)
 - Build (mvn clean compile)
 - Test (mvn test)
 - Package (mvn package)
- Add post actions:
 - Success → print success message with environment
 - Failure → print failure message
- Trigger using Build with Parameters.

Outcome

- Parameterized Jenkins pipeline created
- Code is built, tested, and packaged automatically
- JAR file generated in target/
- Clear visibility of build stages and logs
- Single pipeline works for multiple environments

Advantages

- One pipeline can be reused for different branches and environments
- Reduces manual build and deployment effort
- Early test failures stop bad code from moving forward
- Easy to maintain and extend (SonarQube, Docker, Deploy stages

later)

Prerequisites (Before Starting)

Before creating the Jenkins pipeline, make sure the following are **already available**:

1) Jenkins Server is Running

- Jenkins should be **installed and running**
- Jenkins UI should be accessible in browser
Example:

```
http://<jenkins-ip>:8080
```

-

2) Java Installed on Jenkins Server

- Jenkins needs Java to run Maven builds

Check:

```
java -version
```

Expected:

Java 8 or Java 11

3) Maven Installed on Jenkins Server

- Required to build and package the JAR file

Check:

```
mvn -version
```

4) Git Installed on Jenkins Server

- Needed to clone the Git repository

Check:

```
git --version
```

5) Internet Access from Jenkins Server

- Jenkins must be able to access GitHub:

```
https://github.com/Waseema761/SimpleCustomerApp.git
```

6) Required Jenkins Plugins Installed

Go to:

Manage Jenkins → Plugins

Make sure these are installed:

- ✓ Git Plugin
- ✓ Pipeline Plugin
- ✓ Pipeline: Declarative

7) Git Repository Ready

- Repository contains:
 - `pom.xml`
 - `src/main/java`
 - Maven project structure

Repo used:

`SimpleCustomerApp`

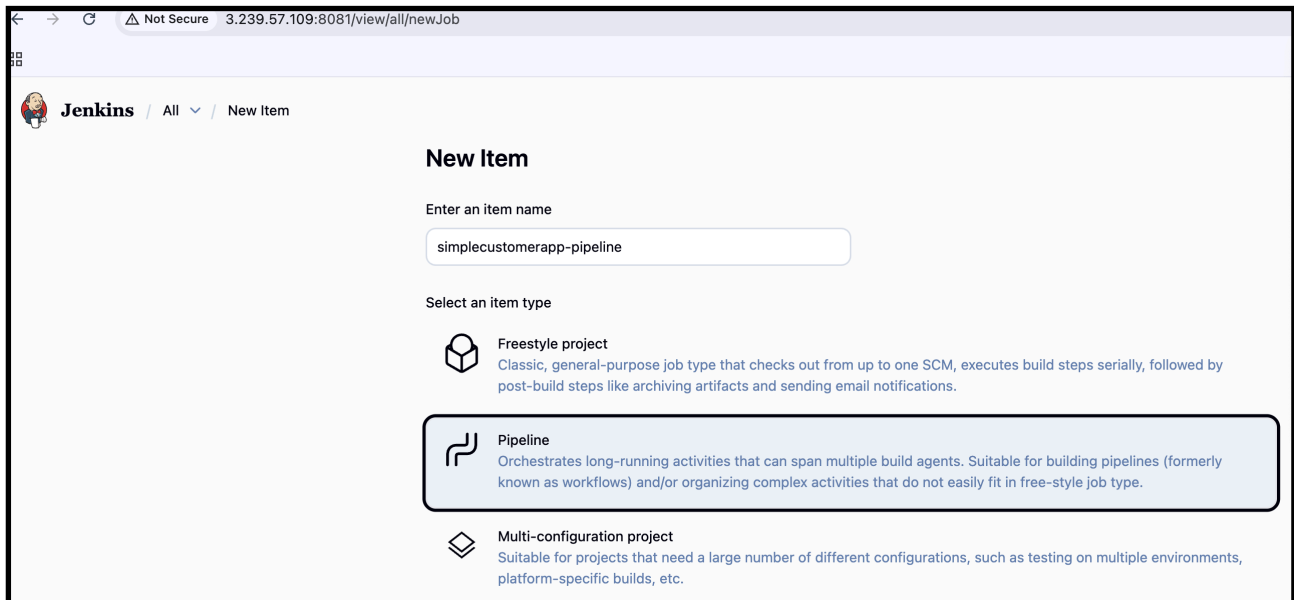
Steps to Create Parameterized Declarative Pipeline

Step 1. Create a New Pipeline Job

1. Open **Jenkins Dashboard**
2. Click **New Item**
3. Enter Job Name:

simplecustomerapp-pipeline

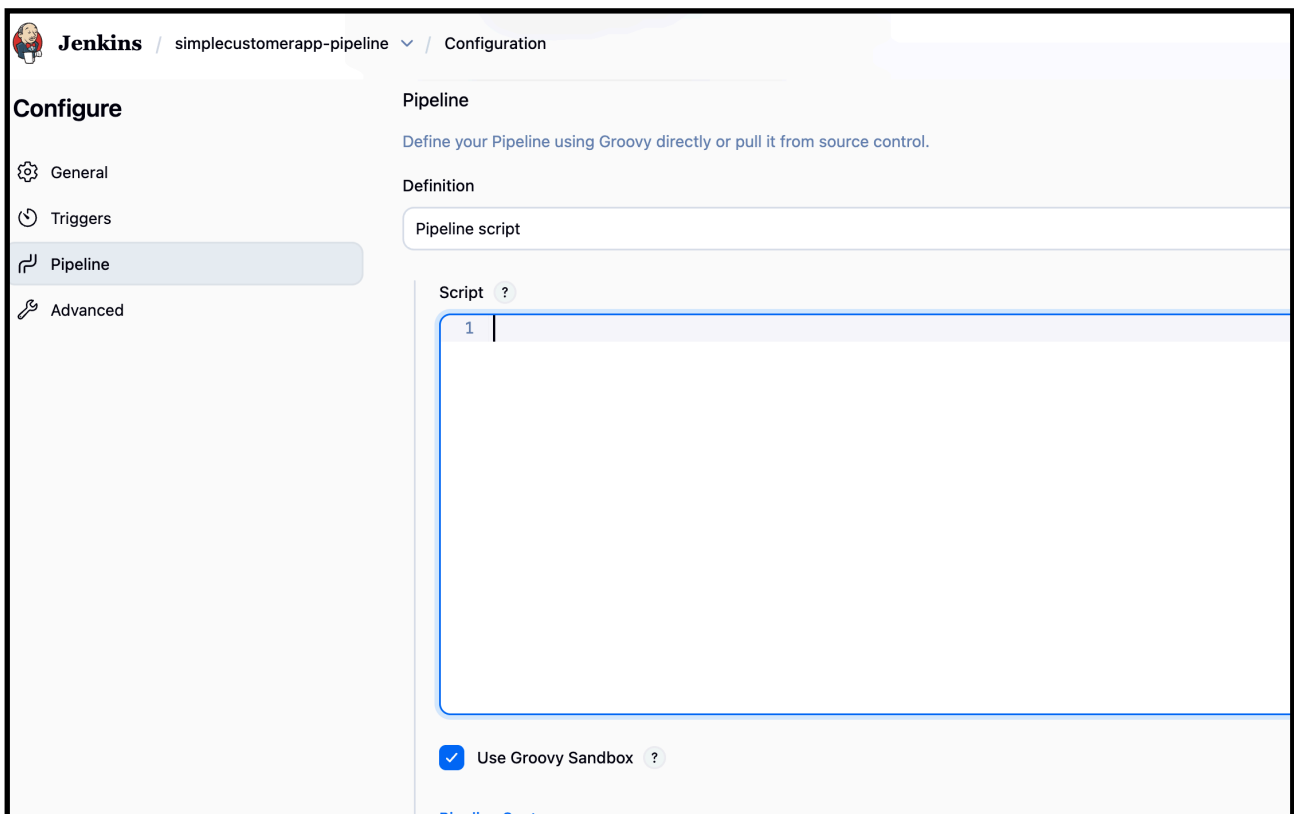
- 4.
5. Select **Pipeline**
6. Click **OK**



Step 2. Configure Pipeline

1. Scroll to **Pipeline** section
2. Choose:

Definition → Pipeline script



Paste the Declarative Jenkins pipeline code: below

```

pipeline {
    agent any

    parameters {
        string(name: 'BRANCH_NAME', defaultValue: 'master')
        choice(name: 'ENV', choices: ['dev', 'qa'])
    }

    environment {
        APP_NAME = 'simplecustomerapp'
    }

    stages {

        stage('Checkout') {
            steps {
                git branch: "${params.BRANCH_NAME}",
                    url: 'https://github.com/Waseema761/
SimpleCustomerApp.git'
            }
        }

        stage('Verify Workspace') {
            steps {
                sh '''
                    pwd
                    ls -l
                    echo "---- inside SimpleCustomerApp ----"
                    cd SimpleCustomerApp
                    pwd
                    ls -l
                    ls -l pom.xml
                '''
            }
        }

        stage('Build') {
            steps {
                dir('SimpleCustomerApp') {
                    sh 'mvn clean compile'
                }
            }
        }
    }
}

```

```

    }

    stage('Test') {
        steps {
            dir('SimpleCustomerApp') {
                sh 'mvn test'
            }
        }
    }

    stage('Package') {
        steps {
            dir('SimpleCustomerApp') {
                sh 'mvn package'
            }
        }
    }
}

post {
    success {
        echo "BUILD SUCCESS for ${APP_NAME} in ${params.ENV}"
        sh 'ls -l SimpleCustomerApp/target/'
    }
    failure {
        echo " BUILD FAILED for ${APP_NAME}"
    }
}
}

```

Step:3

Add Pipeline Parameters

The pipeline is parameterized to support multiple branches and environments.

Parameters used:

- **BRANCH_NAME** (String)
 - Default value: `master`
 - Used to select the Git branch to build

- **ENV (Choice)**
 - Values: **dev**, **qa**
 - Used to represent the target environment

The screenshot shows the Jenkins Pipeline Configuration page. At the top, there's a breadcrumb "pipeline / Configuration". Below it, a checkbox "This project is parameterized" is checked. There are two parameter configuration blocks:

- String Parameter:** Name: `BRANCH_NAME`, Default Value: `master`. It has a "Trim the string" checkbox which is unchecked.
- Choice Parameter:** Name: `ENV`. Choices: `dev`, `qa`.

At the bottom, there are "Save" and "Apply" buttons.

Step 4: Define Environment Variable

An environment variable is defined inside the pipeline:

```
APP_NAME = simplecustomerapp
```

This variable is used in log messages and post actions.

```
8
9  environment {
10      APP_NAME = 'simplecustomerapp'
11  }
12
```


Step 5: Implement Pipeline Stages

5.1 Checkout Stage

- Clones the Git repository
- Checks out the branch provided in `BRANCH_NAME` parameter

Purpose:

- Fetch application source code from GitHub

```
13~ stages {
14~
15~     stage('Checkout') {
16~         steps {
17~             git branch: "${params.BRANCH_NAME}",
18~                 url: 'https://github.com/Waseema761/SimpleCustomerApp.git'
19~         }
20~     }
21~ }
```

5.2 Build Stage

- Runs Maven build command:

```
mvn clean compile
```

-

Purpose:

- Compiles the Java source code
- Ensures there are no compilation errors

```
36~ stage('Build') {
37~     steps {
38~         dir('SimpleCustomerApp') {
39~             sh 'mvn clean compile'
40~         }
41~     }
42~ }
43~ }
```

5.3 Test Stage

- Runs Maven test command:

```
mvn test
```

-

Purpose:

- Executes unit tests
- Ensures code quality before packaging

```
43
44 ✓      stage('Test') {
45 ✓          steps {
46 ✓              dir('SimpleCustomerApp') {
47                  sh 'mvn test'
48              }
49          }
50      }
51
```

5.4 Package Stage

- Runs Maven package command:

```
51
52 ✓      stage('Package') {
53 ✓          steps {
54 ✓              dir('SimpleCustomerApp') {
55                  sh 'mvn package'
56              }
57          }
58      }
59
```

```
mvn package
```

-

Purpose:

- Packages the application into a **JAR file**
- JAR file is generated inside the `target/` directory
-

Step 6: Post Actions

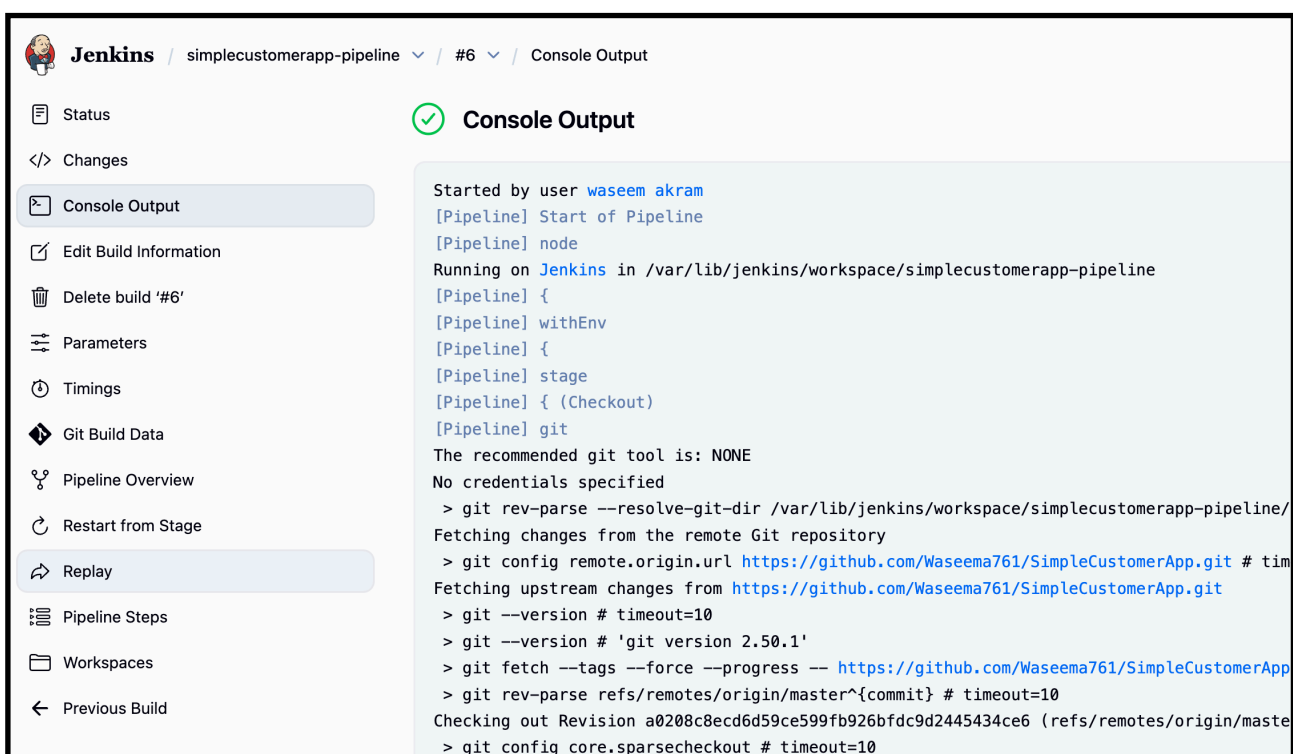
—-> now build the code click build now and check the success /failure

Success Action

- Prints success message with environment name
- Displays the generated JAR file location

Example:

BUILD SUCCESS for simplecustomerapp in dev environment



The screenshot shows the Jenkins web interface for a pipeline named 'simplecustomerapp-pipeline'. The left sidebar contains navigation links: Status, Changes, Console Output (selected), Edit Build Information, Delete build '#6', Parameters, Timings, Git Build Data, Pipeline Overview, Restart from Stage, Replay, Pipeline Steps, Workspaces, and Previous Build. The main area displays the 'Console Output' for build '#6', which is successful (indicated by a green checkmark). The output text shows the pipeline starting, running on Jenkins, and performing a git checkout of revision 'a0208c8ecd6d59ce599fb926bfdc9d2445434ce6' from the remote repository 'https://github.com/Waseema761/SimpleCustomerApp.git'. The build is completed successfully.

```
Jenkins / simplecustomerapp-pipeline / #6 / Console Output

Status
Changes
Console Output
Edit Build Information
Delete build '#6'
Parameters
Timings
Git Build Data
Pipeline Overview
Restart from Stage
Replay
Pipeline Steps
Workspaces
Previous Build

Console Output

Started by user waseem akram
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /var/lib/jenkins/workspace/simplecustomerapp-pipeline
[Pipeline] {
[Pipeline] withEnv
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Checkout)
[Pipeline] git
The recommended git tool is: NONE
No credentials specified
> git rev-parse --resolve-git-dir /var/lib/jenkins/workspace/simplecustomerapp-pipeline/
Fetching changes from the remote Git repository
> git config remote.origin.url https://github.com/Waseema761/SimpleCustomerApp.git # tim
Fetching upstream changes from https://github.com/Waseema761/SimpleCustomerApp.git
> git --version # timeout=10
> git --version # 'git version 2.50.1'
> git fetch --tags --force --progress -- https://github.com/Waseema761/SimpleCustomerApp
> git rev-parse refs/remotes/origin/master^{commit} # timeout=10
Checking out Revision a0208c8ecd6d59ce599fb926bfdc9d2445434ce6 (refs/remotes/origin/maste
> git config core.sparsecheckout # timeout=10
```

Output Verification:

After successful pipeline execution, the JAR file is generated at:

`/var/lib/jenkins/workspace/simplecustomerapp-pipeline/
SimpleCustomerApp/target/`

```
[root@ip-172-31-65-24 ~]# cd /var/lib/jenkins/workspace/
[root@ip-172-31-65-24 workspace]# ls
'MULTI-STAGE PIPELINE'      hiring-app-declarative      job2      pipeline      'scripted pipeline'
'MULTI-STAGE PIPELINE@tmp' hiring-app-declarative@tmp  job3      pipeline@tmp  'scripted pipeline@tmp'
first-job                  hiring-app-parallel         parametirized-job      private      scripted-pipeline
hiring-app                 hiring-app-parallel@tmp     parametirized-job@tmp  private@tmp  scripted-pipeline@tmp
[root@ip-172-31-65-24 workspace]# cd simplecustomerapp-pipeline
[root@ip-172-31-65-24 simplecustomerapp-pipeline]# ls
README.md SimpleCustomerApp SimpleCustomerApp@tmp
[root@ip-172-31-65-24 simplecustomerapp-pipeline]# cd SimpleCustomerApp
-bash: cd: simplecustomerAPP: No such file or directory
[root@ip-172-31-65-24 simplecustomerapp-pipeline]# cd SimpleCustomerApp
[root@ip-172-31-65-24 SimpleCustomerApp]# ls
Service-API.dockerfile WebContent XMLSave.txt build build.xml pom.xml sonar-project.properties src target
[root@ip-172-31-65-24 SimpleCustomerApp]# cd target
[root@ip-172-31-65-24 target]# ls
ls: no input files
[root@ip-172-31-65-24 target]# ls
SimpleCustomerApp-1.0-SNAPSHOT.jar maven-archiver
[root@ip-172-31-65-24 target]# cd
```

In target we are able to see

`simpleCustomerApp-1.0-SNAPSHOT.jar`

Console output :

```
-rw-r--r--. 1 jenkins jenkins 461 Jan  7 07:27 pom.xml
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Build)
[Pipeline] dir
Running in /var/lib/jenkins/workspace/simplecustomerapp-pipeline/SimpleCustomerApp
[Pipeline] {
[Pipeline] sh
+ mvn clean compile
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.javatpoint:SimpleCustomerApp >-----
[INFO] Building SimpleCustomerApp 1.0-SNAPSHOT
[INFO]    from pom.xml
```

```

Running in /var/lib/jenkins/workspace/simplecustomerapp-pipeline/SimpleCustomerApp
[Pipeline] {
[Pipeline] sh
+ mvn test
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.javatpoint:SimpleCustomerApp >-----
[INFO] Building SimpleCustomerApp 1.0-SNAPSHOT
[INFO]    from pom.xml
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- resources:3.3.1:resources (default-resources) @ SimpleCustomerApp ---
[WARNING] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] skip non existing resourceDirectory /var/lib/jenkins/workspace/simplecustomerapp-pipeline/SimpleCustomerApp/src/main/resources
[INFO]
[INFO] --- compiler:3.13.0:compile (default-compile) @ SimpleCustomerApp ---
[INFO] No sources to compile
[INFO]

```

From pom.xml building jar

```

[Pipeline] echo
✅ BUILD SUCCESS for simplecustomerapp in dev
[Pipeline] sh
+ ls -l SimpleCustomerApp/target/
total 4
-rw-r--r--. 1 jenkins jenkins 1441 Jan  7 07:27 SimpleCustomerApp-1.0-SNAPSHOT.jar
drwxr-xr-x. 2 jenkins jenkins  28 Jan  7 07:27 maven-archiver
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS

```

Build succes

Stages :


Jenkins / simplecustomerapp-pipeline ▾ / Stages

Stages

January 7, 2026

✓ #6
12:57 - 10s - 1 change

○ - ✓ - ✓ - ✓ - ✓ - ✓ - ✓ - ○

✗ #5
12:51 - 10s

○ - ✓ - ✓ - ✓ - ✓ - ✗ - ✓ - ○

✗ #4
12:47 - 1.2s

○ - ✓ - ✗ - >> - >> - >> - ✓ - ○

✗ #3

○ - ✓ - ✗ - >> - >> - ✓ - ○

—> pipeline overview :


Jenkins / simplecustomerapp-pipeline ▾ / #6 ▾ / Pipeline Overview

✓ < #6
 Rerun
...

Manually run by wassem akram
 Started 34 min ago
 Queued 1 ms
 Took 10 sec
 <> Changes

Graph
 

Search

✓ Post Actions 0.31s Started 34m ago Jenkins

✓ BUILD SUCCESS for simplecustomerapp in dev 12ms

✓ ls -l SimpleCustomerApp/target/ 0.26s

```

0 + ls -l SimpleCustomerApp/target/
1 total 4
2 -rw-r--r-- 1 jenkins jenkins 1441 Jan 7 07:27 SimpleCustomerApp-1.0-SNAPSHOT.jar
3 drwxr-xr-x 2 jenkins jenkins 28 Jan 7 07:27 maven-archiver
          
```

✓ Checkout 0.23s
 ✓ Verify Workspace 0.29s
 ✓ Build 2.4s
 ✓ Test 2.7s
 ✓ Package 4.0s
 ✓ Post Actions 0.31s


Jenkins / simplecustomerapp-pipeline ▾ / #6 ▾ / Parameters

✓ Parameters

Status
 <> Changes
 Console Output
 Edit Build Information
 Delete build '#6'
 Parameters
 Timings
 Git Build Data

BRANCH_NAME

ENV

Outcome:

- Parameterized Declarative Jenkins pipeline created
- Code is built, tested, and packaged automatically
- JAR file successfully generated
- Clear visibility of pipeline stages and logs
- Same pipeline works for multiple environments

Conclusion:

This task demonstrates the creation of a real-world Jenkins Declarative Parameterized Pipeline.

The pipeline automates code checkout, build, testing, and packaging, and successfully generates a JAR file using Maven.